

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPD801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

12th May 2026



1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π -calculus.

This line of work has been further developed by [Montesi and Peressotti 2021], who provide a unified account reconciling linear logic, the π -calculus, and their metatheory. Their framework serves as the formal basis for the system π LL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, small inconsistencies can arise in the presentation of typing rules within such systems, where the intuitive alignment between variable ordering and type assignment does not precisely match the formal rule structure. These discrepancies are not flaws of the underlying theory, but rather reflect the difficulty of maintaining precision in informal presentations, including in the framework of Montesi and Peressotti.

As observed in discussions with the authors of the framework, such issues are easy to overlook because rule application is often guided by intuition rather than strict syntactic structure. This highlights that even formally correct rules may be unintuitive to apply due to their representation.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanization. Mechanization provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanization in this work is to validate and internalize existing results within a proof assistant. This allows us to eliminate ambiguity arising from informal presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanized developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanized formalization of the π -calculus-based linear logic system π LL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalization of the syntax and typing system of π LL in Lean, including processes, session types, and substitution operations.
- A formalization of the operational semantics for the multiplicative fragment of π LL.
- Formal proofs of session fidelity for the multiplicative fragment.
- A locally nameless representation of binding, eliminating the need for reasoning up to α -equivalence and simplifying substitution reasoning.

Scope of the formalization. This thesis focuses on a mechanized reconstruction of the π LL system of [Montesi and Peressotti 2021], with particular emphasis on its multiplicative fragment. The

syntactic and typing components of πLL have been fully mechanized, including processes, session types, typing judgments, and substitution operations. The operational semantics are defined for the multiplicative fragment, with the exception of the *res*-rule.

Consequently, metatheoretic results concerning typing apply to the full system, while results involving operational semantics are restricted to the multiplicative fragment. The aim of this work is not to extend the theory, but to provide a mechanically verified account of an existing theoretical framework.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of [Montesi and Peressotti 2021]. We cover the full πLL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment πMLL when discussing operational semantics.

Following [Montesi and Peressotti 2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \wp B$, together with the corresponding units 1 and $\perp$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \wp B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B , whereas $A \wp B$ corresponds to receiving a value of type A and proceeding as B . The units 1 and $\perp$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1} \quad \frac{\vdash \Gamma}{\vdash \Gamma, \perp} \perp \quad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B} \otimes \quad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \wp B} \wp$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B , each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B .

The unit rule *rule 1* introduces the unit 1 , which represents the absence of further obligations. It can be understood as an “empty output”, corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\perp$ and $\wp$ describe the behavior of the connective $A \wp B$ and its unit $\perp$. The $\wp$ rule combines resources within a single context, reflecting a form of interaction where both components

are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B . The unit $\perp$ represents the absence of further input, acting as an “empty input” and serving as the dual neutral element.

For example, two independent instances of the unit $\perp$ can be introduced by the [rule 1](#) and combined via [rule \$\otimes\$](#) to derive $\perp \otimes \perp$:

$$\frac{\frac{}{\vdash \perp} \quad \frac{}{\vdash \perp}}{\vdash \perp \otimes \perp} \otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $\perp \otimes \perp$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying π LL.

2.2 The π -Calculus: Processes and Communication

The π -calculus [[Milner et al. 1992](#)] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of [Montesi and Peressotti \[2021\]](#), which adopts [Wadler](#)’s convention of using square brackets ‘[-]’ for output and round parentheses ‘(-)’ for input.

Programs in π MLL are processes $(P, Q, R, \dots)$, which communicate over names $(x, y, z, \dots)$ representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in 2.3. We assume an infinite set of names with decidable equality. The process terms of π MLL are defined by the following grammar:

$P, Q ::=$	$x[y].P$	<i>output y on x and continue as P</i>
	$x(y).P$	<i>input y on x and continue as P</i>
	$x[].P$	<i>output (empty message) on x and continue as P</i>
	$x().P$	<i>input (empty message) on x and continue as P</i>
	$\nu xy.P$	<i>name restriction (cut)</i>
	$P \mid Q$	<i>parallel composition</i>
	$x[L].P$	<i>select left on x and continue as P</i>
	$x[R].P$	<i>select right on x and continue as P</i>
	$x.\text{case}\{L:P, R:Q\}$	<i>offer a binary choice between P or Q on x</i>
	$x[A].P$	<i>output type A on x and continue as P</i>
	$x(X).P$	<i>input a type as X on x and continue as P</i>
	$!x\langle z_1, \dots, z_n \rangle. \{P\}$	<i>server (replicable process) on x depending on servers on $z_1 \dots z_n$</i>
	$x[\text{USE}].P$	<i>consume a server on x and continue as P</i>
	$x[\text{DUP}](x').P$	<i>duplicate server x as x' in P</i>
	$x[\text{DISP}].P$	<i>dispose of a server on x</i>
	$x \leftrightarrow y$	<i>link x and y</i>
	0	<i>terminated process</i>

Term $x[y].P$ allocates a fresh name y , outputs it over x , and proceeds as P . Dually, $x(y).P$ inputs a name over x , binding it to y in P . Terms $x[].P$ and $x().P$ model empty output and input.

Restriction $\nu xy.P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\nu xy.P = \nu yx.P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and 0 is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \nu xz(x[].\mathbf{0} \mid z().y[].\mathbf{0})$. This process creates a private session between the restricted endpoints x and z . The left process, $x[].\mathbf{0}$, signals termination on x before terminating, while the right process, $z().y[].\mathbf{0}$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL . In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In πLL , session types are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types as presented in [Montesi and Peressotti 2021] following [Carbone et al. 2016; Wadler 2014] is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B , while its dual $A \wp B$ describes receiving. The units 1 and $\perp$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \& B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined as follows:

$A, B ::=$	$A \otimes B$	send A , proceed as B		$A \wp B$	receive A , proceed as B
	1	empty output, unit for $\otimes$		$\perp$	empty input, unit for $\wp$
	$A \& B$	offer A or B		$A \oplus B$	select A or B
	$\exists X.A$	existential, type output		$\forall X.A$	universal, type input
	$?A$	client request		$!A$	server accept
	X	type variable		$X^\perp$	dual of type variable

Duality is an involution, $(A^\perp)^\perp = A$, defined structurally on types by swapping each connective with its dual. This ensures that for any well-typed session, the two endpoints always carry dual types, guaranteeing coherent interaction by construction:

$$\begin{aligned}
 (A \otimes B)^\perp &= A^\perp \wp B^\perp & 1^\perp &= \perp & (A \oplus B)^\perp &= A^\perp \& B^\perp \\
 (!A)^\perp &= ?A^\perp & (\exists X.A)^\perp &= \forall X.A^\perp & (X)^\perp &= X^\perp
 \end{aligned}$$

2.4 Environments, Hyperenvironments and Judgements

Typing in πLL is organised into two layers: environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in [Montesi and Peressotti 2021], *environments* $(\Gamma, \Delta, \Xi, \dots)$ are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \dots, x_n : A_n$, where each x_i is associated to its respective A_i , for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then

Γ, Δ is the environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using $'$ as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \quad \Gamma, \Delta = \Delta, \Gamma \quad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by *hyperenvironments* $(\mathcal{G}, \mathcal{H}, \mathcal{I}, \dots)$, which are collections of environments joined using the parallel operator $'|'$. They can be written as follows $\mathcal{G} = \Gamma_1, \dots, \Gamma_n$, where $\Gamma_1, \dots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} | \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using $'|'$ the sum and $\emptyset$ as unit, where $\emptyset$ is the hyperenvironment containing no environments:

$$\mathcal{G} | \emptyset = \mathcal{G} \quad \mathcal{G} | \mathcal{H} = \mathcal{H} | \mathcal{G} \quad (\mathcal{G} | \mathcal{H}) | \mathcal{I} = \mathcal{G} | (\mathcal{H} | \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations $(\mathcal{D}, \mathcal{E}, \mathcal{F}, \dots)$ are written as $\vdash P :: \mathcal{G}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P , appearing in the judgement concluding a derivations is well-typed.

Multiplicative Fragment

$$\begin{array}{c} \frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^\perp}{\vdash \nu x y P :: \mathcal{G} \mid \Gamma, \Delta} \text{CUT} \quad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}} \text{MIX} \quad \frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \\ \frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B} \otimes \quad \frac{\vdash P :: \emptyset}{\vdash x[] . P :: x : \mathbf{1}} \mathbf{1} \quad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \wp B} \wp \quad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \perp} \perp \end{array}$$

Additional rules for Additives, Quantifiers and Exponentials

$$\begin{array}{c} \frac{\vdash P :: \Gamma, x : A}{\vdash x[L].P :: \Gamma, x : A \oplus B} \oplus_1 \quad \frac{\vdash P :: \Gamma, x : B}{\vdash x[R].P :: \Gamma, x : A \oplus B} \oplus_2 \quad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{L:P, R:Q\} :: \Gamma, x : A \& B} \& \\ \frac{}{\vdash x \leftrightarrow y :: x : A^\perp, y : A} \text{AX} \quad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A} ? \quad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A} \text{w} \quad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A} \text{c} \\ \frac{\vdash P :: z_1 : ?B_1, \dots, z_n : ?B_n, x : A}{\vdash !x(z_1, \dots, z_n). \{P\} :: z_1 : ?B_1, \dots, z_n : ?B_n, x : !A} ! \quad \frac{\vdash P :: \Gamma, x : B \{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B} \exists \quad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B} \forall \end{array}$$

Fig. 1. π LL, typing rules.

The multiplicative rules form the core of the system. **Rule CUT** composes two processes in P sharing dual endpoints $x : A$ and $y : A^\perp$, hiding them via restriction. This is the logical equivalent of communication. **Rule MIX** and **rule MIX₀** compose independent processes in parallel and introduce the terminated process, respectively.

Rule $\otimes$ and **rule $\wp$** type output and input. Note that **rule $\otimes$** types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, **rule $\wp$** keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The units **rule 1** and **rule $\perp$** type empty output and empty input, where **rule 1** requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to 0 .

The additive rules (**rule $\oplus_1$** , **rule $\oplus_2$** , and **rule $\&$**) type selection and offering of alternative processing paths. Note that **rule $\&$** requires both branches to be typed in the *same* context Γ , reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The **rule $!$** requires all free names except x to be typed using $?$, enforcing that a server only depends on other servers. The **rule $?$** , **rule w** , and **rule c** allow a client to use, discard, or duplicate a server respectively.

The **rule ax** types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. **Rule $\exists$** and **rule $\forall$** type polymorphic communication, where the side condition $X \notin \text{ft}(\Gamma)$ in **rule $\forall$** , ensures the introduced type variable, X , does not accidentally shadow one already existing in Γ .

Example 2.2. The process from [Example 2.1](#) has the typing $\vdash vxz(z().y[].0 \mid x[].0) :: y:1$, as shown by the derivation below:

$$\begin{array}{c}
 \frac{\frac{\frac{}{\vdash 0 :: \emptyset} \text{MIX}_0}{\vdash x[].0 :: x:1} 1 \quad \frac{\frac{\frac{}{\vdash 0 :: \emptyset} \text{MIX}_0}{\vdash y[].0 :: y:1} 1}{\vdash z().y[].0 :: y:1, z:\perp} \perp}{\vdash x[].0 \mid z().y[].0 :: x:1 \mid y:1, z:\perp} \text{MIX} \\
 \frac{}{\vdash vxz(x[].0 \mid z().y[].0) :: y:1} \text{CUT}
 \end{array}$$

Note that, for the typing context $x:1 \mid y:1, z:\perp$ to align with the premise of **rule CUT** , we utilize the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \emptyset$ and $\Gamma = \bullet$. Under these instantiations, the premise of **rule CUT** : $\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp$ can be viewed as $x:A \mid \Delta, y:A^\perp$. This effectively instantiates the **rule CUT** over the dual types $A = 1$ and $A^\perp = \perp$ on channels x and y , with $\Gamma = y:1$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for a creating a valid **rule CUT** (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in [Figure 2](#). The full LTS is given in [Montesi and Peressotti \[2021\]](#).

Remark 2.3. The operational semantics rely explicitly on the structural equivalence rule (**rule $=_\alpha$**) to rename restricted variables (via $=_\alpha$) prior to a transition. In paper-based proofs, α -equivalence is often treated implicitly or swept under the rug via Barendregt's Variable Convention. However, in a mechanized setting, explicitly reasoning about α -equivalence over named variables is notoriously difficult and introduces overwhelming boilerplate, as variable capture must be manually avoided during substitution [??]. As we will see in ??, this challenge motivates the use of a Locally Nameless

Fig. 2. π MLL, transition rules for derivations.

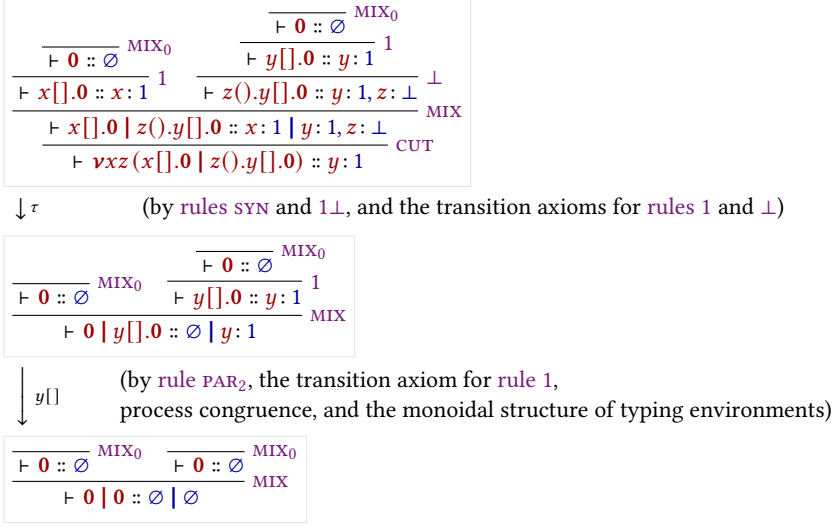


Fig. 3. An execution for the derivation in Example 2.2.

representation for the syntax of πLL , which structurally enforces α -equivalence and natively resolves the complexities of bound variable renaming.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l \mid l') \mid l, l' \in Act, i(l) \cap i(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

Example 2.4. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (**1**, **1**, **⊗**, **⊗**) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule **syn** allows for the pairing of concurrent actions via $l \mid l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules **1** and **⊗** simplify the term into an internal τ -transition, modeling the synchronization of the session.

2.6 Process Semantics and Erasure

While the typing derivations of πLL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π -calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti 2021] and define an erasure function, $proc : Der \rightarrow Proc$, which strips away the typing information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed πLL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$\begin{array}{c}
\frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0}{\vdash v[].\mathbf{0} :: v:1} \perp}{\vdash w().v[].\mathbf{0} :: v:1, w:\perp} \perp \quad \frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0}{\vdash x[].\mathbf{0} :: x:1} \perp \quad \frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0}{\vdash y[].\mathbf{0} :: y:1} \perp}{\vdash z().y[].\mathbf{0} :: y:1, z:\perp} \perp}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x:1 \mid y:1, z:\perp} \text{MIX} \\
\frac{\vdash w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0} :: v:1, w:\perp \mid x:1 \mid y:1, z:\perp}{\vdash \nu xz (w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v:1, w:\perp \mid y:1} \text{CUT}
\end{array}$$

Fig. 4. A derivation for the process $\nu xz w().y[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}$

Example 2.5. Consider the well-typed process P formed by adding a third parallel component to the derivation of [Example 2.2](#) before the cut (A derivation of which can be seen on [Figure 4](#)):

$$P = \nu xz (w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$$

The component containing w and v is independent from the component containing x , z , and y . This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$\begin{array}{ccc}
P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0} & P \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{w()} \xrightarrow{v[]} \mathbf{0} \\
P \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{\tau} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0}
\end{array}$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated in [[Montesi and Peressotti 2021](#)], this relationship is established functorially. Let

$$\mathcal{P}(X) = \{X' \mid X' \subseteq X\} \text{ and } \mathcal{L}(X) = X \times Lbl$$

be endofunctors representing the states and labeled transitions, respectively. The erasure function $proc$ acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields a standard Operational Correspondence (or Erasure) theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

THEOREM 2.6 (OPERATIONAL CORRESPONDENCE). *Let $\mathcal{D}$ be a valid typing derivation such that $proc(\mathcal{D}) = P$.*

- (1) **Soundness of Erasure:** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*
- (2) **Completeness of Erasure:** *If $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.*

This theorem allows us to define the operational target for π LL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

Remark 2.7. The untyped LTS for processes relies heavily on explicit side conditions regarding free and introduced names to prevent the collision of bound variables (e.g., ensuring $i(l) \cap i(l') = \emptyset$). However, when operating at the level of typing derivations, these side conditions become inherently redundant. Because π LL enforces linear resource management, the structural rules of the typed

$$\begin{array}{c}
\begin{array}{c}
x[x'].P \xrightarrow{x[x']} P \quad x(x').P \xrightarrow{x(x')} P \quad x[] .P \xrightarrow{x[]} P \quad x().P \xrightarrow{x()} P \\
\frac{P \xrightarrow{l} P' \quad i(l) \cap f(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q} \text{PAR}_1 \quad \frac{Q \xrightarrow{l} Q' \quad i(l) \cap f(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'} \text{PAR}_2 \\
\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad i(l \parallel l') \cap f(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l \parallel l'} P' \mid Q'} \text{SYN} \quad \frac{P \xrightarrow{x[x'] \parallel y(y')} P'}{vxy P \xrightarrow{\tau} vxy vx'y' P'} \otimes \otimes \\
\frac{P \xrightarrow{x[] \parallel y()} P'}{vxy P \xrightarrow{\tau} P'} 1\bot \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{vxy P \xrightarrow{l} vxy P'} \text{RES} \quad \frac{P =_\alpha Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_\alpha
\end{array}
\end{array}$$

$$\begin{array}{c}
i(l \parallel l') = i(l) \cup i(l') \\
f(l \parallel l') = f(l) \cup f(l') \\
i(\tau) = \emptyset \\
f(\tau) = \emptyset \\
i(x[]) = i(x()) = \emptyset \\
f(x[]) = f(x()) = \{x\} \\
i(x[x']) = i(x(x')) = \{x'\} \\
f(x[x']) = f(x(x')) = \{x\}
\end{array}$$

Fig. 5. π MLL, transition rules for processes.

LTS dictate that hyperenvironments can only be merged when their domains are strictly disjoint. Consequently, any well-typed derivation inherently satisfies the name-distinctness and freshness requirements of the underlying untyped calculus, shifting the burden of safety from dynamic transition checks to static structural typing.

PROPOSITION 2.8. *The operational semantics of π LL processes strictly respects the introduction of names via transition labels. Formally, if $P \xrightarrow{l} P'$, then:*

- $i(l) \cap f(P) = \emptyset$
- If P is well-typed, then $i(l) = f(P') \setminus f(P)$

Remark 2.9. Proposition 2.8 is vital for mechanizing the metatheory of π LL. Because proof assistants tools require explicit tracking of fresh and bound variable to prevent accidental capture, this proposition acts as the formal bridge between the dynamic semantics and the static typing. It guarantees that a well-typed process evolves predictably, ensuring that it does not spontaneously generate resources during a transition.

2.7 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In π LL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow [Montesi and Peressotti 2021] and define a function $env : Der \rightarrow Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, Montesi and Peressotti [Montesi and Peressotti 2021] derive a LTS specifically for typing environments (lts_e).

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x : \perp$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalized by the τ -reflexivity axiom:

$$\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$$

This axiom states that any valid hyperenvironment transitions to itself under a silent τ -action.

Example 2.10. Consider a process typed by the environment $\mathcal{G} = x : \perp, y : \perp, z : 1$. If the process performs the observable action $x()$, the environment must mimic this action, consuming the type

for x :

$$x : \perp, y : \perp, z : 1 \xrightarrow{x()} y : \perp, z : 1$$

If the process subsequently performs an internal τ -synchronization, the environment remains completely unchanged:

$$y : \perp, z : 1 \xrightarrow{\tau} y : \perp, z : 1$$

The requirement that a process can only perform actions permitted by its typing environment is known as **Session Fidelity**. Just as the *proc* function established operational correspondence for processes, the *env* function establishes a correspondence for environments.

THEOREM 2.11 (SESSION FIDELITY). *Let $\mathcal{D}$ be a valid typing derivation such that $\text{env}(\mathcal{D}) = \mathcal{G}$ and $\text{proc}(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition $\mathcal{G} \xrightarrow{l} \mathcal{G}'$, and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.*

Remark 2.12. Session Fidelity is the ultimate touchstone for the safety of a session-typed language. However, proving this in a proof assistant introduces distinct challenges. While rules like τ -reflexivity are trivial on paper, mechanizing the invariant that ‘an environment does not change’ requires robust structural equivalence proofs for contexts. Furthermore, properly tracking the consumption and preservation of types dynamically necessitates a rigorous representation of bound variables and freshness, which heavily dictates the architectural choices discussed in the following chapters.

2.8 The Burden of Bound variables

References

- Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.
- Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23–26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33
- Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/fall2023/entries/logic-linear/>
- Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4
- Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.
- Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π -calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818
- Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.